CrocSort — GPU-Accelerated Sorting

what it is: a system that offloads sorting to the GPU using radix sort. works as a drop-in for pyarrow/polars/numpy, and we also tested it for database index creation and external sort on TPC-H. runs on a single RTX 6000 over PCIe 3.0.

hardware: Quadro RTX 6000 (24 GB, Turing), 2x Xeon Gold (48 threads), 192 GB DDR4, PCIe 3.0

---

how it works

the basic idea: convert sort keys to a byte-comparable format (sign-flip + big-endian), ship them to the GPU over PCIe, run CUB radix sort (O(N) instead of O(N log N)), get back a permutation, and reorder the original data.

for in-memory columnar data (arrow tables, numpy arrays, etc.) this is the whole pipeline. the GPU sort kernel itself takes ~70ms for 300M rows — most of the time is actually spent on PCIe transfer and encoding, not sorting.

for datasets bigger than GPU memory (like TPC-H SF100 at 72 GB), we chunk the data, sort each chunk on the GPU, and do a K-way merge on CPU. we also do a "compact key" trick where we detect which byte positions actually vary across the dataset and only upload those — saves 2-8x on PCIe transfer for real data.

key encoding supports: int8/16/32/64, uint8/16/32/64, float32/64, date32, fixed-width strings.

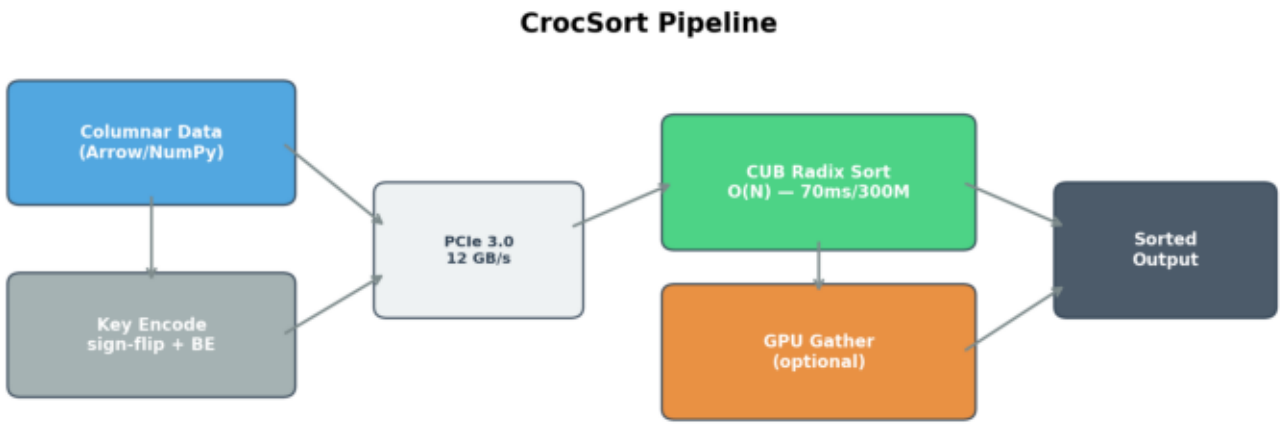

Figure 1: CrocSort pipeline — encode keys on CPU, sort on GPU, gather results

results — columnar analytics engines

tested against the main python sort engines on 100M int32 rows.
GPU sort is 2-89x faster depending on the baseline.

the more interesting comparison is polars, which is the fastest CPU engine — we still get 2.2x on single-column and 9.5x on multi-column sorts.

at 300M rows with 4 sort columns (Arrow C++ benchmark), the full GPU pipeline (encode + upload + sort + gather) takes 4.3s vs pyarrow's 97s — about 22x.

the actual GPU sort is only 9% of our time; most is key encoding (55%) and PCIe upload (29%). this means faster interconnects (PCIe 5, NVLink C2C) would help a lot.

the crossover point where GPU starts winning is around 5,000 rows.

multi-column and strings

multi-column sorts show bigger speedups because CPU comparison-sort cost grows with key width, while GPU radix sort cost grows linearly (just more bytes to upload).

string sort works by encoding to fixed-width null-padded byte keys. UUIDs (36B) get 38x speedup, email addresses (variable-length, avg ~24B) get 26x.

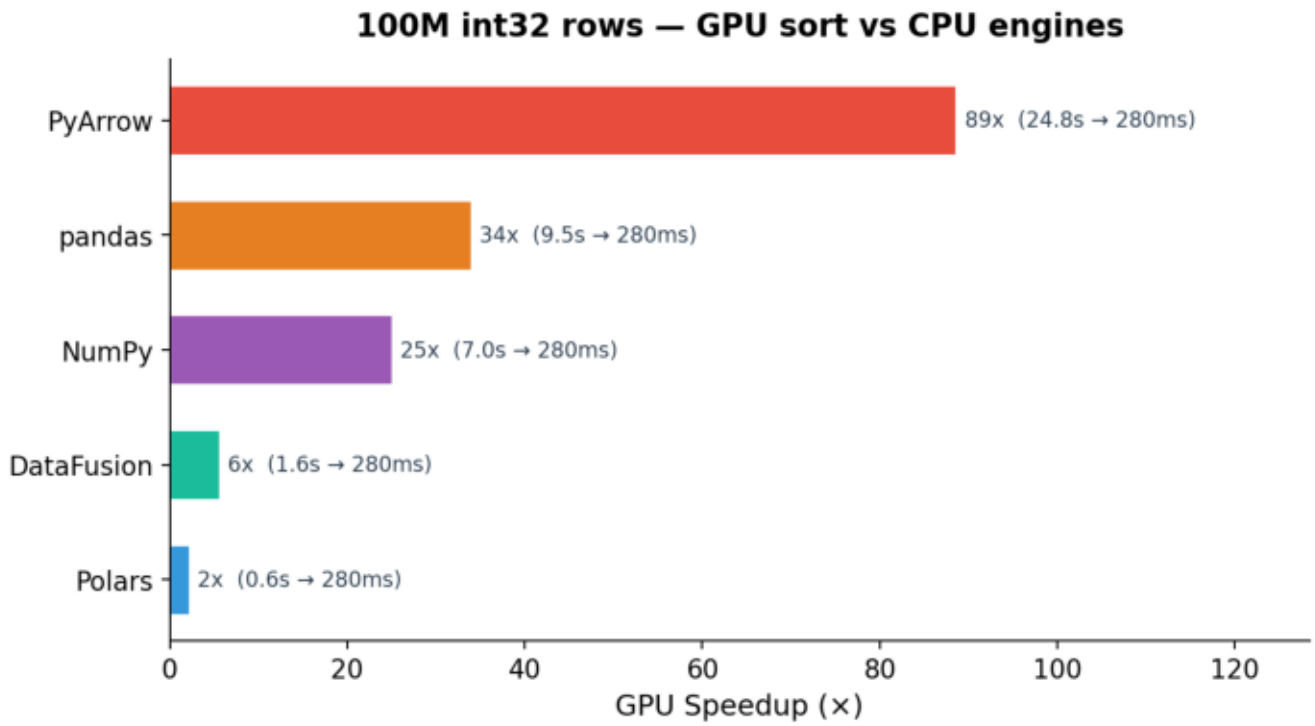

Figure 2: GPU speedup vs CPU engines at 100M rows

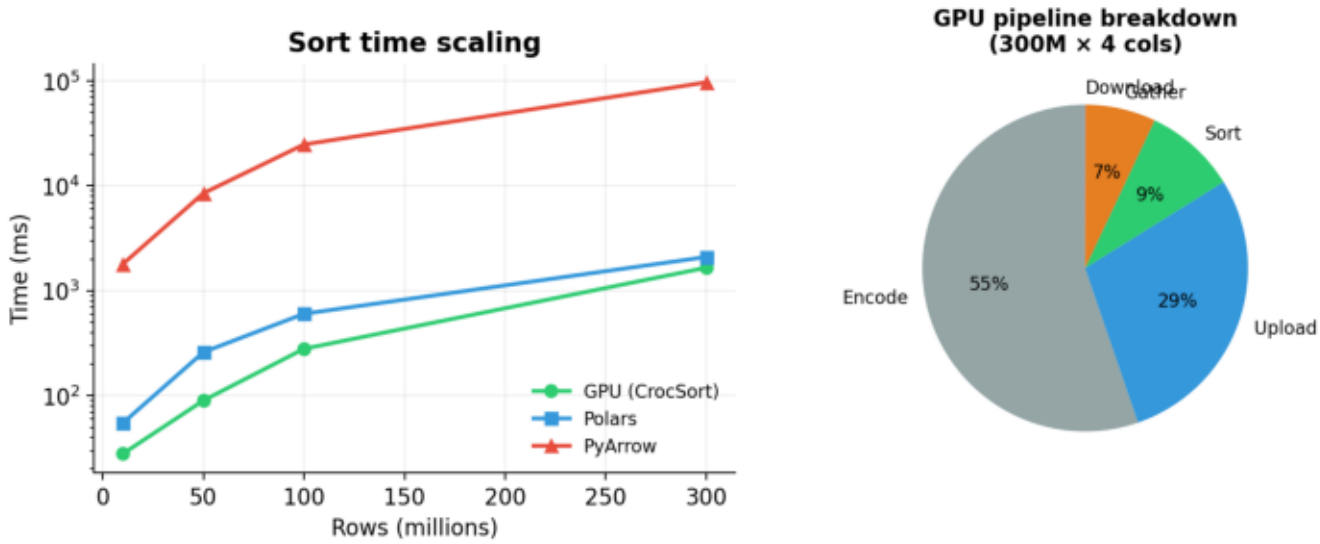

Figure 3: Scaling from 10M to 300M rows (left); GPU time breakdown at 300M (right)

results — database index creation

this is probably the most compelling application. CREATE INDEX is basically just a big sort, and databases do it single-threaded (or poorly parallelized).

benchmarked against SQLite — 100M rows: 87.9s vs 0.72s on GPU = 122x faster. PostgreSQL numbers from literature are similar (~52s for 100M rows single-threaded, ~10 min for 500M rows).

this matters operationally: index creation locks the table. if you can do it in under a second instead of a minute, you don't need maintenance windows for schema migrations.

also relevant for LSM-tree compaction (RocksDB/LevelDB) — prior work (LUDA, 2020) already showed 2x throughput by offloading the sort to GPU.

results — external sort (TPC-H)

full multi-column sort of TPC-H lineitem (9 sort columns, 66-byte records).
4.7-10x faster than DuckDB depending on scale factor.

SF100 (600M rows, 72 GB) sorts in 8 seconds at ~9 GB/s throughput.
for context, MendSort (JouleSort 2023 winner) does 3.3 GB/s on GenSort,
and we hit 8 GB/s on wider records with a single GPU.

we also tried integrating directly into DuckDB as a custom operator, but the row-serialization overhead (DuckDB converts columns to rows for sorting) ate all the GPU advantage — end-to-end query time was basically 1.0x. this is why we pivoted to the columnar approach where data is already in the right format.

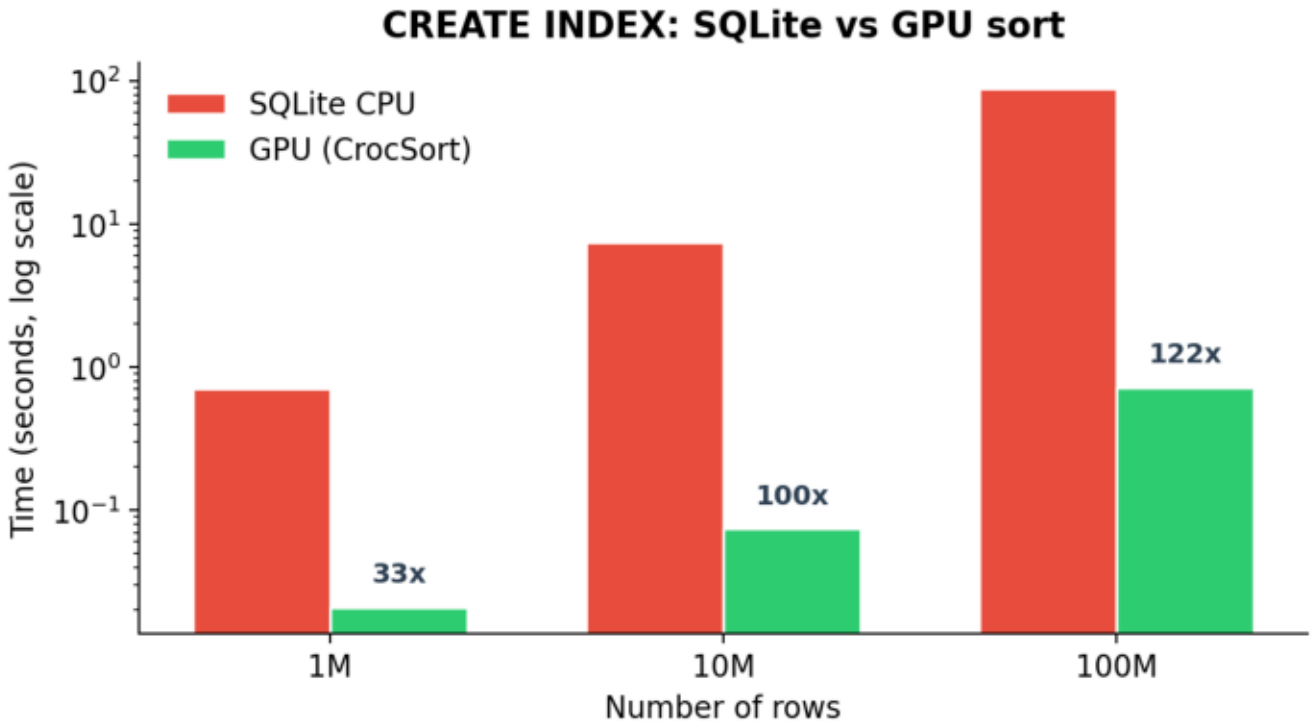

Figure 4: CREATE INDEX — SQLite vs GPU sort (log scale)

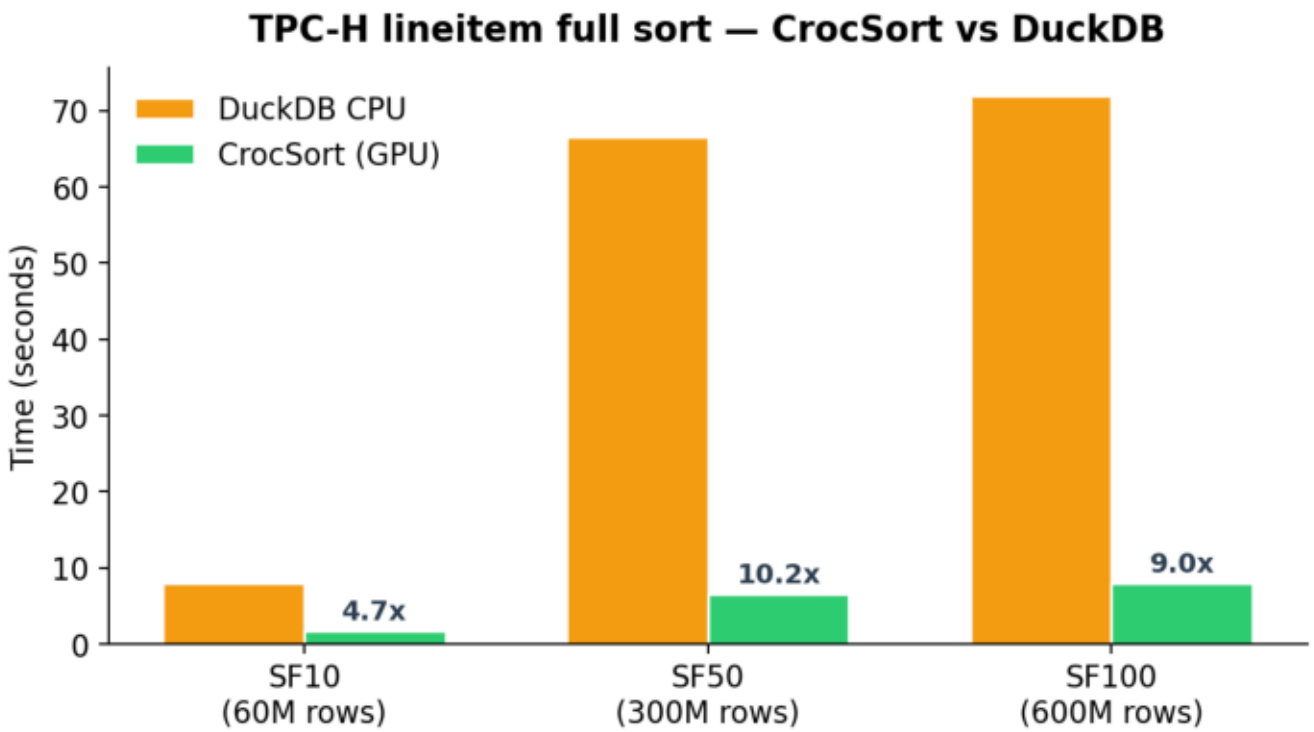

Figure 5: TPC-H lineitem full sort — CrocSort vs DuckDB

hardware projections

the current system is bottlenecked by PCIe 3.0 (12 GB/s) and host DRAM bandwidth.

the big one is GH200 (Grace Hopper) — its 900 GB/s NVLink C2C between CPU and GPU means the gather phase (26% of time on RTX 6000) goes to zero. the GPU writes sorted output directly into unified memory. projected ~1.2s for SF50 vs current 6.6s.

what's next

- postgres integration — replace the sort in tuplesort_performsort() for CREATE INDEX. postgres already normalizes keys to byte-comparable format so this is straightforward.
- rocksdb compaction — GPU sort in background compaction to reduce write stalls.
- GH200 testing — unified memory should give ~5x over current numbers.
- multi-GPU — NVLink all-to-all for larger datasets.

where it makes sense (and where it doesn't)

good fit:

- columnar analytics (pyarrow, polars, datafusion) — 2-89x, drop-in replacement
- CREATE INDEX in postgres/mysql/sqlite — 30-122x, eliminates maintenance windows
- LSM compaction (rocksdb) — background sort, no PCIe latency on query path
- any bulk sort > 5,000 rows

bad fit:

- per-query OLTP sorts (< 100K rows) — PCIe latency dominates
- row-oriented engines (DuckDB) — serialization overhead eats the GPU advantage

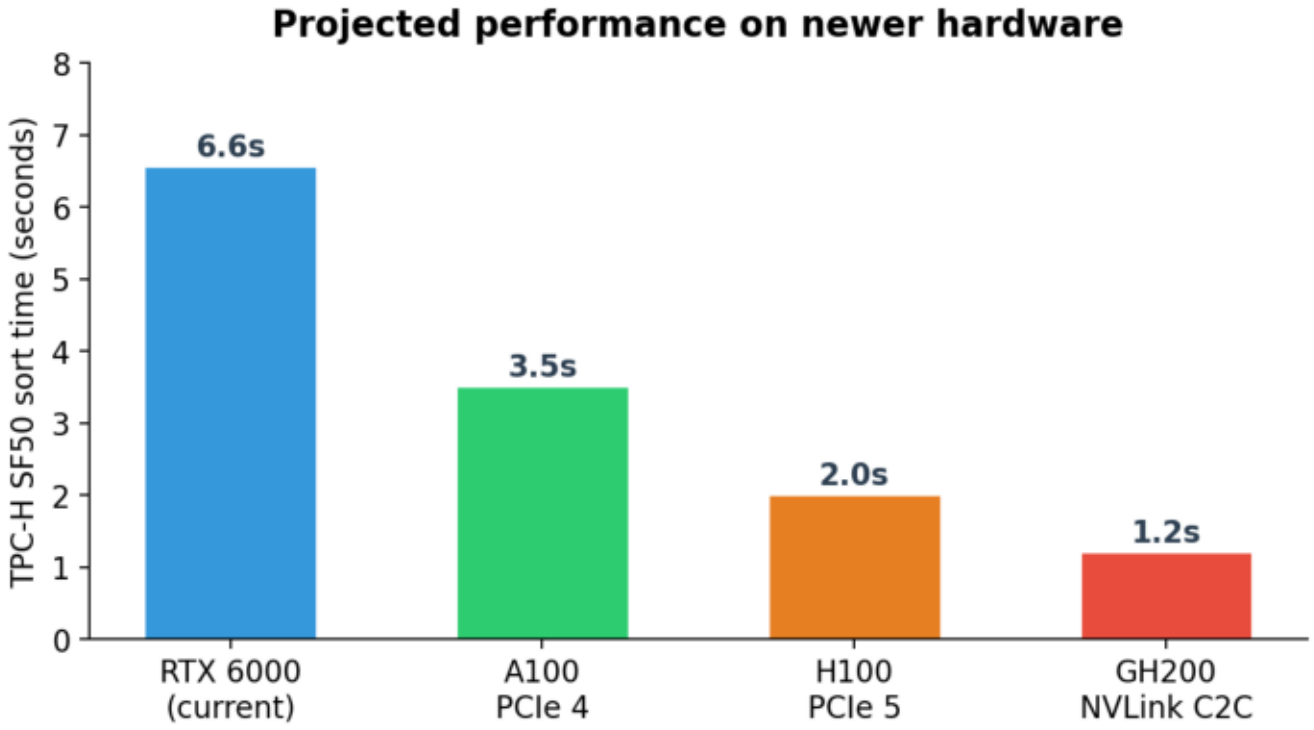

Figure 6: Projected sort time on newer GPU hardware (TPC-H SF50)

Workload	CPU Baseline	GPU (CrocSort)	Speedup
100M int32 sort	0.60s (Polars)	0.28s	2.2x
100M 2-col int32	3.68s (Polars)	0.39s	9.5x
300M int32 (Arrow C++	96.8s	4.3s	22x
100M numpy int32	7.0s	0.28s	25x
10M UUIDs (36B strings	4.46s (Polars)	0.12s	38x
10M emails (var-len)	3.64s (Polars)	0.14s	26x
100M CREATE INDEX	87.9s (SQLite)	0.72s	122x
TPC-H SF50 (300M rows	66.6s (DuckDB)	6.56s	10x
TPC-H SF100 (600M row	~72s (DuckDB)	8.02s	9x
60 GB GenSort (100B re	N/A	7.5s	8 GB/s throughput